

Agent Memory Architecture

5 Layers That Cut Token Cost 90% and Make Your Agent Actually Learn

Based on CoALA cognitive architecture, Mem0, Anthropic memory systems, Snowflake ontology research, and LangChain production engineering materials
Independently compiled, September 2026 — not affiliated with Anthropic — and not endorsed

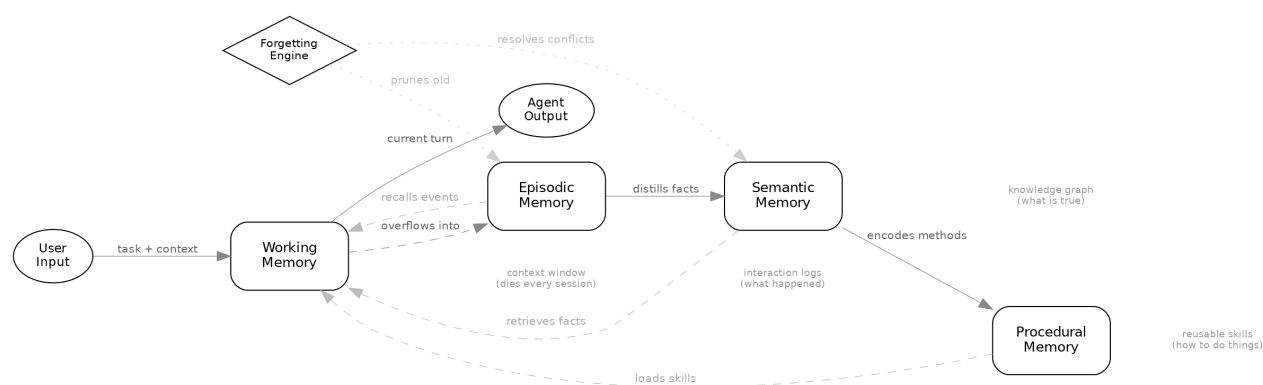


Fig. 1. Agent memory flow architecture. User input enters Working Memory (context window). When context fills, it overflows into Episodic Memory (logs). Episodic memory distills facts into Semantic Memory (knowledge graph). Semantic memory encodes methods as Procedural Memory (skills). The Forgetting Engine prunes stale data and resolves contradictions. Dashed arrows show retrieval paths back to Working Memory.

Abstract — Your agent reads the same 12 files every morning, reaches the same conclusions, and charges you the same tokens for the privilege. The model got 10x smarter in two years. It still forgets everything after every session. This note presents the 5-layer memory architecture that solves this: working memory (the context window), episodic memory (what happened), semantic memory (what is true), procedural memory (how to do things), and forgetting (what to delete). We compile evidence from Mem0 (90% token reduction, 91% lower latency), Snowflake (20% accuracy gain from one ontology layer), and production systems showing that memory — not the model — is the bottleneck in 2026. The playbook provides implementation code, templates, decision frameworks, and a production checklist.

Index Terms — Agent memory, working memory, episodic memory, semantic memory, procedural memory, forgetting, knowledge graph, context window, CoALA, Mem0.

I. THE PROBLEM: AMNESIA AT SCALE

Why the smartest model in the world forgets everything

Every model call starts with an empty context window. The previous session's discoveries,

decisions, corrections, and preferences are gone. The agent that spent 40 minutes debugging a migration script yesterday does not remember the script exists today. It will read the same files, try the same approaches, hit the same dead ends, and bill you the same tokens [1].

The industry invested heavily in making models smarter — larger context windows, better reasoning, faster inference. But intelligence without memory is Groundhog Day. A brilliant colleague who forgets every conversation is not a colleague. They are an expensive consultant you must re-brief every morning.

The CoALA framework (Cognitive Architectures for Language Agents) provides the vocabulary: working memory, episodic memory, semantic memory, procedural memory [2]. These are not metaphors borrowed from psychology. They are

engineering layers, each with distinct storage, retrieval, and expiration characteristics. Production agents need all five layers — including a forgetting engine that prevents contradictions from accumulating.

A. The Cost of Forgetting

TABLE I — THE COST OF FORGETTING

Metric	Without Memory	With Memory	Source
Tokens per query	26,000	1,800	Mem0 [3]
Latency	Baseline	-91%	Mem0 [3]
Accuracy	Baseline	+20%	Snowflake [4]
Tool calls	Baseline	-39%	Snowflake [4]
Context rebuilds	Every session	Never	Architecture

II. LAYER 1: WORKING MEMORY

The context window — everything the agent sees right now

Working memory is the context window. It contains the system prompt, the current conversation, tool results, retrieved documents, and any injected state. When the context fills up, old content is either truncated or compressed. This is where most agents stop — and it is exactly why they forget [2].

A. What Lives in Working Memory

TABLE II — WORKING MEMORY CONTENTS

Content	Purpose	Lifespan
System prompt	Agent identity and rules	Every call
Current conversation	Task context	Current session
Tool results	Grounded data	Current task
Retrieved documents	RAG context	Current query
Injected state	Memory summaries	Current session

B. The Overflow Problem

When the context window fills, the agent must choose what to keep and what to discard. Most

systems truncate from the beginning — removing the oldest messages first. This destroys the task definition, early decisions, and the reasoning that led to the current state. The better approach: overflow important content into episodic memory before it is lost.

Listing 1 — Context Overflow Handler

```
def handle_overflow(context, episodic_store, budget):
    if len(context.tokens) > budget * 0.85:
        # Save important content before truncation
        decisions = extract_decisions(context)
        facts = extract_facts(context)
        episodic_store.save(decisions, facts)
        # Compact context, keep recent + saved refs
        context.compact(keep_recent=budget*0.6)
    return context
```

III. LAYER 2: EPISODIC MEMORY

What happened — interaction logs with timestamps and outcomes

Episodic memory records what happened during agent interactions: which tasks were attempted, what approaches were tried, which succeeded, which failed, and why. It is the agent's experience log. Without it, the agent repeats mistakes and re-discovers solutions [2].

A. What to Store

Not everything belongs in episodic memory. Store: task descriptions and outcomes, tool calls and their results, errors and their resolutions, decisions and their reasoning, user corrections and preferences. Do not store: verbose reasoning chains (compress them), raw document contents (reference by ID), or intermediate drafts (keep only the final version and the key revision points).

Listing 2 — Episodic Memory Record

```
@dataclass
class Episode:
    task_id: str
    timestamp: datetime
    task_description: str
    approach: str
```

```

outcome: str # success | failure | partial
key_decisions: list[str]
errors_encountered: list[str]
resolution: str
user_feedback: str | None
tokens_used: int
duration_seconds: int

```

B. Retrieval Strategy

When a new task arrives, the agent should query episodic memory for similar past tasks before starting work. Similarity can be computed by task description embedding, tool overlap, or explicit tags. The retrieved episodes tell the agent: this task is similar to task-042, which failed because the migration script had a typo in line 47. Skip directly to the fix. This single retrieval saves the entire debugging session [3].

C. Expiration

Episodic memory without expiration grows indefinitely. Old episodes become irrelevant — the codebase changed, the API was updated, the user's preferences evolved. Set a retention policy: keep the last 90 days by default, archive episodes with high-value outcomes permanently, expire routine episodes after 30 days. Let the user override with explicit pins.

IV. LAYER 3: SEMANTIC MEMORY

What is true — facts, entities, and relationships as a knowledge graph

Semantic memory stores durable facts about the world: "user prefers TypeScript," "the API endpoint is /v2/users," "the production database is on AWS us-east-1." These facts do not expire when the session ends. They are the agent's accumulated knowledge — distilled from episodic experiences into a queryable store [2].

A. Implementation: From JSON to Graph

TABLE III — SEMANTIC MEMORY STORAGE OPTIONS

Storage	Best For	Scales To	Query Power
JSON file	Prototyping, <100 facts	Small	Key lookup
SQLite	Single-user, <10K facts	Medium	SQL queries
Vector DB	Similarity search	Large	Embedding search
Knowledge graph	Multi-hop reasoning	Large	Graph traversal

B. The Ontology Layer

Snowflake's research showed that adding a single ontology layer — a schema that defines entity types, relation types, and validation rules — improved agent accuracy by 20% and reduced tool calls by 39% [4]. The ontology is not complex. It is a simple definition: users have names and preferences, projects have owners and deadlines, APIs have endpoints and authentication methods. The schema constrains what the agent stores and how it queries, preventing the knowledge base from becoming a dump of unstructured observations.

Listing 3 — Minimal Ontology Schema

```

ontology = {
  "entity_types": ["user", "project", "api", "config"],
  "relation_types": ["prefers", "owns", "depends_on",
    "located_at"],
  "validation": {
    "user.prefers": {"target": "string", "expires":
      "90d"},
    "project.owns": {"target": "user", "required": True},
    "api.located_at": {"target": "string", "required":
      True},
  }
}

```

C. Conflict Resolution

When the agent learns that the user now prefers Python over TypeScript, it must update the semantic memory — not append a second preference. Conflict resolution is the operation that keeps semantic memory consistent: detect duplicates, merge compatible facts, supersede outdated ones, and flag genuine contradictions for human review. Without conflict resolution, semantic memory accumulates contradictions that

produce incoherent agent behavior.

V. LAYER 4: PROCEDURAL MEMORY

How to do things — methods that worked, saved as reusable skills

Procedural memory stores methods. The agent tried three approaches to deploy the frontend. One worked. That approach becomes a reusable skill: a named method with steps, tools, preconditions, and success criteria. Next time the task appears, the agent loads the skill and skips straight to what worked [2].

A. Skill Structure

Listing 4 — Procedural Memory Skill

```
@dataclass
class Skill:
    name: str
    trigger: str # when to use this skill
    steps: list[str]
    tools_required: list[str]
    preconditions: list[str]
    success_criteria: str
    failure_modes: list[str]
    version: int
    last_used: datetime
    success_rate: float
```

B. Skill Promotion

Not every successful approach deserves to become a skill. Promote a method only when: the task recurs at least weekly, the method has succeeded at least three times, the steps can be described without ambiguity, and the tools required are reliably available. A single lucky success is an episode, not a skill. The bar for promotion prevents procedural memory from filling with brittle, context-dependent methods.

C. Skill Versioning

When the deployment process changes, the old skill becomes invalid. Version skills like software: update the steps, increment the version number,

keep the previous version available for rollback, and re-validate against the current environment. A stale skill is worse than no skill — it produces confident wrong behavior.

VI. LAYER 5: FORGETTING

What to delete — the most important layer nobody builds

An agent that never forgets accumulates contradictions. The user moved from New York to London, but the agent still recommends restaurants in Brooklyn. The API migrated from v1 to v2, but the agent still calls v1 because both endpoints exist in semantic memory. The deploy process changed, but the agent still follows the old skill because it has a higher success rate from historical runs [5].

A. Forgetting Is Not a Bug

In human cognition, forgetting is a feature. It prevents irrelevant memories from overwhelming relevant ones. It resolves contradictions by letting outdated facts decay. It reduces cognitive load by pruning information that no longer matters. Agent memory systems need the same mechanism — a deliberate forgetting engine that prunes, supersedes, and resolves [5].

B. Three Forgetting Operations

TABLE IV — FORGETTING OPERATIONS

Operation	What It Does	When to Use
Expiration	Remove facts older than threshold	Periodic cleanup
Supersession	Replace old fact with new version	When facts change
Contradiction resolution	Flag conflicting facts for review	When both facts are current

Listing 5 — Forgetting Engine

```
def forget(memory, rules):
    # Expire old episodes
    for ep in memory.episodic:
        if ep.age > rules.episodic_ttl:
            memory.archive(ep)
```

```
# Supersede outdated semantic facts
for fact in memory.semantic:
    newer = memory.find_newer(fact)
    if newer:
        fact.status = 'superseded'
        fact.superseded_by = newer.id
# Flag contradictions
conflicts = memory.find_conflicts()
for c in conflicts:
    memory.flag_for_review(c)
```

A. Mem0: 90% Token Reduction

Mem0 (51K+ GitHub stars) implements a memory layer that sits between the application and the LLM. Instead of stuffing the entire conversation history into every API call, Mem0 extracts and stores relevant facts in a persistent store. When the next query arrives, only the relevant facts are injected — typically 1,800 tokens instead of 26,000. The result: 90% fewer tokens per query and 91% lower latency. The memory layer paid for itself within the first day of deployment for every production user who reported metrics [3].

B. Snowflake: Ontology Beats Brute Force

Snowflake's AI engineering team tested agent performance with and without an ontology layer on their internal data platform. The ontology defined entity types (tables, columns, users, queries), relation types (owns, depends_on, queries), and validation rules. With the ontology, agent accuracy improved by 20% and tool calls decreased by 39%. The ontology did not add information the agent did not have. It organized existing information so the agent could find and use it efficiently [4].

C. Anthropic: Scoped Memory Isolation

Anthropic's Claude implements memory with strict project-scoped isolation. Facts stored in one project's memory are invisible to other projects. This prevents cross-contamination: a coding agent's preferences do not leak into a writing agent's style. The architecture treats memory as a first-class security boundary, not just a convenience feature.

Each memory store has its own read/write permissions, expiration policies, and audit trail [7].

D. The Frontier Agent Gap

Research on SWE-bench and similar benchmarks shows that frontier models reach only 62.2% on real long-horizon tasks requiring more than 20 tool calls. The primary failure mode is not reasoning — it is context loss. The agent forgets what it discovered in step 3 by the time it reaches step 15. Multi-step tasks require memory, and memory requires architecture, not just a bigger context window.

E. Common Memory Anti-Patterns

TABLE IV-B — MEMORY ANTI-PATTERNS

Anti-Pattern	Why It Fails	Fix
Store everything	Noise drowns signal	Store only decisions and outcomes
Never expire	Contradictions accumulate	30-90 day TTL + pins for critical
No schema	Store becomes dump	Add ontology with types + validation
No retrieval	Memory exists but unused	Query before every task
One big store	Cross-project leaks	Scope memory per project/domain
No versioning	Stale skills cause errors	Version skills, keep rollback
Storing raw context	Expensive, noisy, large	Extract facts, compress context

F. The Memory Tax

Memory is not free. Every retrieval adds latency. Every stored fact consumes storage. Every forgetting operation requires computation. The architecture must justify itself: if the token savings from memory retrieval exceed the cost of maintaining the memory system, the architecture pays for itself. Mem0's numbers (90% token reduction) show this is achievable, but only with disciplined storage — not by saving everything.

G. Template: Memory Configuration File

Template 1 — Memory System Config

```
MEMORY_CONFIG:
  episodic:
    store: ./memory/episodes.jsonl
    max_entries: 1000
```

```

ttl_days: 90
retrieve_top_k: 5
semantic:
  store: ./memory/facts.db
  ontology: ./memory/schema.yaml
  conflict_policy: keep_newer
  audit_stale_days: 90
procedural:
  store: ./memory/skills.yaml
  promotion_threshold: 3
  version_history: 5
forgetting:
  schedule: daily
  expire_episodes: 30d
  supersede_on_conflict: true
  flag_contradictions: true

```

H. Template: Memory-Aware System Prompt

Template 2 — System Prompt with Memory

You have access to a persistent memory system.
Before starting any task:

1. Query episodic memory for similar past tasks
2. Load relevant semantic facts into context
3. Check if a procedural skill exists for this task type

After completing any task:

1. Save the outcome to episodic memory
2. Extract any new durable facts to semantic memory
3. If this approach worked 3+ times, propose it as a skill

Never assume you remember something from a previous session.

Always check memory. Memory is the source of truth, not your training.

VII. THE COMPLETE ARCHITECTURE

How all five layers work together

TABLE V — THE 5-LAYER ARCHITECTURE

Layer	Stores	Retrieval	Expiration	Implementation
Working	Current context	Always loaded	End of call	Context window
Episodic	What happened	By similarity	30-90 days	Log store
Semantic	What is true	By entity/query	On supersession	Graph/DB
Procedural	How to do things	By trigger match	On version update	Skill store
Forgetting	What to delete	Scheduled	Continuous	Cleanup engine

The layers interact through a data flow, not through independent storage. When working memory overflows, important content flows into episodic memory. When episodic patterns repeat, they distill into semantic facts. When semantic facts describe a reliable method, they encode as procedural skills. The forgetting engine prunes all

three long-term layers to prevent staleness and contradiction.

VIII. BUILD PATH: ONE WEEK TO PRODUCTION MEMORY

Start with the cheapest layer that solves the biggest pain

TABLE VI — 7-DAY BUILD PATH

Day	Build	Exit Test
1	Add episodic log: save task outcome to JSON file	Agent recalls yesterday's failure
2	Add retrieval: query log before starting task	Agent ignores known dead ends
3	Add semantic store: extract durable facts	Agent follows user preferences
4	Add ontology: define entity/relation types	Agent validates against schema
5	Add procedural skills from repeated successes	Agent uses correct method
6	Add forgetting: expire old episodes, supersede facts	Agent updates fact store
7	Wire overflow: context → episodic → semantic	End-to-end

A. Day 1-2: Episodic Memory First

Start with the simplest possible episodic memory: append every task outcome to a JSON file. Before starting a new task, read the last 10 entries and inject a summary into the context. This alone prevents the agent from repeating yesterday's mistakes. The implementation cost is one file write and one file read per session.

B. Day 3-4: Semantic Memory

Extract durable facts from episodic memory: user preferences, project configurations, API endpoints, team conventions. Store them in a simple key-value format or SQLite table. Before starting work, inject relevant facts into the context. The agent now remembers that the user prefers TypeScript without being told every session.

C. Day 5-6: Skills and Forgetting

When the same task-type succeeds three times with the same approach, save the approach as a named

skill. When facts conflict, keep the newer one and mark the older as superseded. When episodes age past 30 days, archive them. The system now learns, remembers, and cleans up after itself.

D. Day 7: Wire the Full Pipeline

Connect all layers: context overflow triggers episodic save, episodic patterns trigger semantic extraction, semantic methods trigger skill creation, and the forgetting engine runs daily. Test by running a multi-session task that requires remembering decisions from day 1 on day 7. If the agent remembers without re-reading the original files, the pipeline works.

VIII-B. IMPLEMENTATION PATTERNS

Production code for each memory layer

A. Episodic Memory with JSONL

The simplest episodic store is a JSONL file — one JSON object per line, appended after each task. JSONL is append-only by nature, which means no corruption from concurrent writes. Each line contains the episode record from Listing 2. To retrieve relevant episodes, read the last N entries or filter by keyword in the task description.

Listing 6 — Episodic Store Implementation

```
class EpisodicStore:
    def __init__(self, path='memory/episodes.jsonl'):
        self.path = path

    def save(self, episode):
        with open(self.path, 'a') as f:
            f.write(json.dumps(asdict(episode))+'\n')

    def retrieve(self, query, top_k=5):
        episodes = [json.loads(l) for l in open(self.path)]
        scored = [(ep, similarity(query,
            ep['task_description']))
            for ep in episodes]
        scored.sort(key=lambda x: x[1], reverse=True)
        return [ep for ep, _ in scored[:top_k]]
```

B. Semantic Memory with SQLite

For production semantic memory, SQLite provides durability, querying power, and zero infrastructure. The schema defines entity types, relation types, and fact records. Each fact has a source (which episode produced it), a timestamp, and a status (active, superseded, or flagged). This is the minimum viable knowledge graph.

Listing 7 — Semantic Store Schema

```
CREATE TABLE facts (
    id INTEGER PRIMARY KEY,
    entity TEXT NOT NULL,
    entity_type TEXT NOT NULL,
    relation TEXT NOT NULL,
    value TEXT NOT NULL,
    source_episode TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status TEXT DEFAULT 'active',
    superseded_by INTEGER REFERENCES facts(id)
);
```

C. Procedural Memory with YAML

Skills are stored as YAML files — one per skill. The file contains the trigger condition, the steps, the required tools, and the success criteria. The agent loads all skill triggers at startup and matches them against incoming tasks. When a match is found, the full skill is loaded into working memory as an instruction set.

Listing 8 — Skill File Format

```
name: deploy-frontend
trigger: 'deploy frontend to production'
version: 3
last_used: 2026-09-01
success_rate: 0.92
steps:
  - run: npm run build
  - verify: build/index.html exists
  - run: aws s3 sync build/ s3://prod-bucket
  - verify: curl https://app.example.com returns 200
preconditions:
  - AWS credentials configured
  - npm dependencies installed
failure_modes:
  - build failure: check node version
  - s3 permission denied: check IAM role
```

D. The Full Memory Pipeline

Listing 9 — Memory-Aware Agent Loop

```
def agent_with_memory(task, memory):
    # 1. Check procedural memory for matching skill
    skill = memory.procedural.match(task)
    if skill and skill.success_rate > 0.8:
        return execute_skill(skill, task)
```

```

# 2. Check episodic memory for similar past tasks
episodes = memory.episodic.retrieve(task, top_k=3)

# 3. Load relevant semantic facts
facts = memory.semantic.query(task.entities)

# 4. Inject into working memory and execute
context = build_context(task, episodes, facts)
result = llm.call(context)

# 5. Save outcome to episodic memory
memory.episodic.save(Episode(task, result))

# 6. Extract new facts to semantic memory
new_facts = extract_facts(result)
memory.semantic.store(new_facts)

# 7. Check for skill promotion
if memory.episodic.count_successes(task.type) >= 3:
    memory.procedural.promote(task.type)

return result

```

IX. DECISION FRAMEWORK

When to add each memory layer

TABLE VII — MEMORY DECISION FRAMEWORK

Problem	Add This Layer	Do Not Add Yet
Agent repeats yesterday's mistakes	Episodic memory	Knowledge graph
Agent forgets user preferences	Semantic memory (key-value)	Full ontology
Agent re-discovers same solution	Procedural memory	Skill marketplace
Old facts contradict new ones	Forgetting engine	Full versioning system
Agent re-reads same files every session	Semantic + episodic	RAG pipeline
Token cost too high	Memory summaries (Mem0 plugin)	Custom embeddings

X. PRODUCTION CHECKLIST

TABLE VIII — PRODUCTION READINESS CHECKLIST

#	Requirement	Failure If Missing
1	Episodic log saves task outcomes	Agent repeats mistakes
2	Episodic retrieval before new tasks	Agents ends re-explored
3	Semantic store for durable facts	Preferences forgotten
4	Ontology schema validates entities	Ontology becomes dump
5	Conflict resolution for semantic facts	Contradictions accumulate
6	Procedural skills from repeated methods	New methods re-discovered
7	Skill versioning with rollback	Stale skills produce errors
8	Expiration policy for episodes	Unbounded storage growth
9	Context overflow handler	Important content truncated
10	Forgetting engine runs on schedule	Old data pollutes decisions

XI. LIMITATIONS

A. Memory Is Not Understanding

Storing facts does not guarantee the agent uses them correctly. A semantic memory entry "user prefers TypeScript" does not help if the agent cannot translate that preference into code-generation decisions. Memory augments the model's reasoning; it does not replace it. The memory architecture makes more information available. The model must still reason about it.

B. Forgetting Can Lose Important Context

A too-aggressive expiration policy deletes episodes that contain crucial one-time decisions. A too-conservative policy accumulates noise. The correct balance depends on the domain: a medical agent should retain episodes for years; a coding agent can expire most episodes after 30 days. Let users pin important episodes that should never expire.

C. Semantic Memory Requires Maintenance

Facts change. APIs move. Teams reorganize. Preferences evolve. A semantic memory that was accurate in January may be dangerously wrong in September. Schedule regular audits: compare stored facts against current reality. Flag facts that have not been confirmed in 90 days. Let users correct facts directly.

XI-B. MEMORY FOR MULTI-AGENT SYSTEMS

When multiple agents share a workflow

When multiple agents collaborate, memory becomes a coordination mechanism. Agent A discovers a fact. Agent B needs that fact 10 minutes later. Without shared memory, Agent B re-discovers it from scratch. With shared semantic memory, Agent B queries the store and finds the fact immediately. The memory system replaces entire categories of inter-agent communication [6].

A. Shared vs. Private Memory

TABLE VII-B — MEMORY SCOPING FOR MULTI-AGENT SYSTEMS

Memory Scope	Visible To	Use Case
Private episodic	Only the owning agent	Internal reasoning history
Private semantic	Only the owning agent	Agent-specific preferences
Shared semantic	All agents in workflow	Project facts, API endpoints
Shared procedural	All agents in workflow	Common deployment methods
Global semantic	All agents in system	Organization-wide config

B. The Reader-Writer Problem

When two agents update the same semantic fact simultaneously, one write overwrites the other. The memory system needs a concurrency strategy: last-write-wins (simple but lossy), merge-on-conflict (complex but complete), or single-writer-per-entity (simple and safe). For most production systems, single-writer-per-entity is the correct default: each fact is owned by one agent, and other agents read but do not write.

C. Memory as the Shared Brain

In the SpaceXAI Playbook architecture, the Manager Bot maintains a ledger of all tasks, owners, states, and deadlines. This ledger IS the shared semantic memory. Specialist bots read it to understand what work is available and write their results back. The ledger replaces conversation-based coordination: agents do not message each other. They read and write to a shared store. The result is a system that scales without the

N-squared communication problem of direct agent-to-agent messaging.

XI-C. WHEN NOT TO ADD MEMORY

Not every agent needs persistent memory

TABLE VII-C — WHEN TO ADD MEMORY

Scenario	Add Memory?	Why
One-off tasks (format this text)	No	Nothing to remember
Daily report from same data	Yes	Remember format preferences
Customer support triage	Yes	Remember customer history
Creative brainstorming	No	Freshness matters more than history
Multi-session debugging	Yes	Must remember what was tried
Batch data processing	No	Stateless by design
Personal assistant	Yes	Preferences, calendar, contacts
Code review	Yes	Remember past review patterns

The test: would the agent produce a better result if it remembered what happened last time? If yes, add memory. If the task is stateless by design (batch processing, one-off formatting), memory adds complexity without value. If the task benefits from accumulated knowledge (debugging, personal assistance, customer support), memory is the single highest-ROI investment you can make.

XI-D. MEMORY SYSTEMS COMPARISON

TABLE VII-D — MEMORY SYSTEMS LANDSCAPE

System	Type	Stars	Key Feature
Mem0	Memory layer	51K+	90% token reduction
Zep	Long-term memory	3K+	Temporal knowledge graph
LangMem	LangChain memory	1K+	Hot/warm/cold storage
Claude Memory	Built-in	N/A	Project-scoped isolation
Custom JSON	DIY	N/A	Full control, zero deps

The simplest production path: start with custom JSON (day 1), validate the architecture works (days 2-7), then migrate to Mem0 or Zep when you need scale, search, and managed infrastructure. Building

on a framework before validating the pattern wastes integration time on an architecture that may not fit your workflow.

XIII. ADVANCED MEMORY PATTERNS

Techniques for production-scale memory systems

A. Hierarchical Summarization

When episodic memory grows beyond thousands of entries, raw retrieval becomes noisy. Hierarchical summarization compresses old episodes into summaries at increasing time scales: daily summaries from individual episodes, weekly summaries from daily summaries, monthly summaries from weekly summaries. The agent queries the most recent detailed episodes plus the compressed summaries of older periods. This provides both recent precision and long-range context without unlimited storage growth [3].

Listing 10 — Hierarchical Summarization

```
def summarize_hierarchy(epsodes, llm):
    # Group by day
    by_day = group_by_date(epsodes)
    daily_summaries = []
    for day, day_eps in by_day.items():
        if len(day_eps) > 5:
            summary = llm.summarize(day_eps)
            daily_summaries.append(DailySummary(
                date=day, summary=summary,
                episode_count=len(day_eps),
                key_outcomes=[e.outcome for e in day_eps]
            ))
        # Archive raw episodes
        for ep in day_eps:
            ep.status = 'archived'
    return daily_summaries
```

B. Memory-Aware Routing

In multi-agent systems, memory can drive task routing. When a new task arrives, the router queries episodic memory to find which agent has the most experience with similar tasks. The agent with the highest success rate on similar past tasks receives the assignment. This is implicit specialization

through memory — agents develop expertise not through explicit role assignment but through accumulated experience [6].

Listing 11 — Memory-Aware Router

```
def route_by_memory(task, agents, memory):
    best_agent = None
    best_score = 0
    for agent in agents:
        past = memory.episodic.retrieve(
            query=task.description,
            agent_id=agent.id, top_k=5)
        success_rate = sum(
            1 for ep in past if ep.outcome == 'success'
        ) / max(len(past), 1)
        if success_rate > best_score:
            best_score = success_rate
            best_agent = agent
    return best_agent or agents[0] # default to first
```

C. Contradiction Detection Pipeline

Contradictions in semantic memory are silent killers. The agent stores "deploy target: staging" and later stores "deploy target: production" without superseding the first entry. Both facts are active. The agent may use either one depending on retrieval order. A contradiction detection pipeline runs periodically, compares all active facts for the same entity-relation pair, and either auto-resolves (keep newer) or flags for human review.

TABLE X — CONTRADICTION TYPES AND RESOLUTION

Conflict Type	Detection	Auto-Resolution	Human Review
Same entity, different values	Exact match on entity	Keep newer	If values are close
Negation	Semantic similarity + Negation	Keep both	Always
Scope conflict	One general, one specific	Keep specific	If scopes overlap
Temporal	Same fact, different time	Keep most recent	If both are current

D. Hot/Warm/Cold Storage Tiers

Not all memories need the same access speed. Hot memory (in-context, instant access) should contain: current task state, active user preferences, and the last 3 relevant episodes. Warm memory (one retrieval step, 100-500ms) should contain: all episodic logs from the last 30 days, active semantic facts, and current procedural skills. Cold memory (archive, 1-5s) should contain: old episodes,

superseded facts, and deprecated skills. This tiering reduces both latency and cost [8].

TABLE XI — HOT/WARM/COLD STORAGE TIERS

Tier	Access Time	Storage	Contents	Cost
Hot	0ms (in context)	Context window	Current state, top-10 episodes	Deposited
Warm	100-500ms	SQLite / Mem0	30-day episodes, active facts	Active
Cold	1-5s	S3 / Archive	Old episodes, superseded facts	Archived

XIV. MEMORY BY DOMAIN

Different agents need different memory configurations

TABLE XII — MEMORY CONFIGURATION BY AGENT TYPE

Agent Type	Primary Layer	Retention	Key Fact Types
Coding agent	Procedural + Episodic	90 days	Build cmds, patterns, failures
Customer support	Semantic + Episodic	1 year	Customer history, preferences
Research agent	Semantic + Episodic	Permanent	Sources, findings, methods
Personal assistant	Semantic	Permanent	Preferences, contacts, schedule
DevOps agent	Procedural + Semantic	30 days	Deploy steps, infra state
Sales agent	Episodic + Semantic	1 year	Deal history, contacts, notes

A. Coding Agents

Coding agents benefit most from procedural memory. The deploy process, the test commands, the linting rules, the code patterns that pass review — these are methods that should be saved as skills after three successful uses. Episodic memory captures debugging sessions: what was tried, what failed, what fixed it. Semantic memory stores project facts: which framework, which version, which patterns are banned. The forgetting engine expires episodes older than 90 days but preserves skills indefinitely.

B. Customer Support Agents

Customer support agents need long-term semantic memory. The customer's name, account type, previous issues, preferred communication style, and

escalation history must persist across months of interactions. Episodic memory records each interaction: what the customer asked, what was resolved, what was escalated. The critical difference from coding agents: retention must be much longer (1 year minimum), and the forgetting engine must be conservative — deleting a customer's history creates a worse experience than keeping stale data.

C. Research Agents

Research agents need permanent semantic memory. Sources, findings, hypotheses, contradictions, and connections between papers form a growing knowledge graph that becomes more valuable over time. Episodic memory records which papers were read, what was extracted, and which search queries produced useful results. Procedural memory captures research methods: how to search, how to evaluate, how to synthesize. The forgetting engine should never expire semantic facts in a research context — supersede outdated findings instead.

XV. TESTING YOUR MEMORY SYSTEM

How to verify that memory actually works

A. The Amnesia Test

Tell the agent a fact in session 1. Close the session. Open session 2 and ask the agent to recall the fact. If it cannot, episodic or semantic memory is not working. This is the minimum viable test for any memory system.

B. The Contradiction Test

Store two conflicting facts: "user prefers Python" and "user prefers TypeScript." Ask the agent which language the user prefers. If it confidently answers

one without acknowledging the conflict, the contradiction detection is not working. The correct behavior: flag the conflict and ask the user to resolve it.

C. The Staleness Test

Store a fact with a 30-day TTL. Wait (or simulate) 31 days. Ask the agent about the fact. If it still uses the expired fact, the forgetting engine is not running. The correct behavior: the fact should be expired and the agent should indicate it does not have current information.

D. The Skill Promotion Test

Complete the same task type successfully three times. On the fourth attempt, verify that the agent uses the saved skill instead of re-planning from scratch. If it re-plans, procedural memory is not triggering. The correct behavior: the agent loads the skill and follows its steps, only deviating if the preconditions are not met.

E. The Load Test

Fill episodic memory with 10,000 entries. Measure retrieval latency. If retrieval takes more than 500ms, the store needs indexing or the retrieval algorithm needs optimization. For semantic memory, insert 5,000 facts and verify that conflict detection still runs in under 1 second. Memory systems that work at 100 entries and fail at 10,000 are not production-ready.

Memory System Test Checklist

- Agent recalls fact from previous session (amnesia test)
- Agent detects conflicting facts (contradiction test)
- Expired facts are not used (staleness test)
- Repeated success triggers skill (promotion test)
- Retrieval under 500ms at 10K entries (load test)
- Forgetting engine runs on schedule (cron test)
- Multi-session task preserves progress (continuity test)
- Cross-project facts do not leak (isolation test)

XII. CONCLUSION

The model got 10x smarter. It still forgets everything after every session. Memory — not intelligence — is the bottleneck in 2026. The five-layer architecture solves this: working memory for the current turn, episodic memory for what happened, semantic memory for what is true, procedural memory for how to do things, and a forgetting engine for what to delete.

The evidence is concrete: Mem0 cut token usage by 90% and latency by 91% by storing 1,800 tokens per query instead of 26,000. Snowflake added one ontology layer and gained 20% accuracy with 39% fewer tool calls. Memory pays for itself on day one.

The practical recommendation: start with episodic memory. One JSON file that saves task outcomes and queries them before starting new work. This costs minutes to build and immediately prevents the agent from repeating yesterday's mistakes. Then add semantic memory for durable facts. Then procedural skills for methods that work. Then a forgetting engine to keep everything clean. The architecture compounds: each layer makes the others more valuable.

"Tomorrow it reads the same twelve files, reaches the same conclusions, and charges you the same tokens. Unless you give it memory."

APPENDIX: KEY TERMS

TABLE IX — GLOSSARY

Term	Definition
Working memory	Context window: everything the agent sees this turn
Episodic memory	Interaction logs: what happened, when, and the outcome
Semantic memory	Durable facts: entities, relationships, preferences
Procedural memory	Reusable skills: methods that worked, saved for reuse
Forgetting engine	Cleanup system: expire, supersede, resolve contradictions

Term	Definition
Ontology	Schema defining entity types and relation types
CoALA	Cognitive Architectures for Language Agents framework
Skill promotion	Converting a repeated success into a reusable procedure
Supersession	Replacing an outdated fact with a newer version

REFERENCES

[1] LangChain, "Agent Memory Systems for Production," blog.langchain.com, 2026.

[2] T. Sumers et al., "Cognitive Architectures for Language Agents (CoALA)," 2024.

[3] Mem0, "Memory Layer for AI Agents," mem0.ai, 2026. 51K+ GitHub stars.

[4] Snowflake, "Ontology-Enhanced Agent Memory," [Snowflake Engineering](https://snowflake.com), 2026.

[5] A. Richards, "Why Your Agent Needs to Forget," [AI Engineering Blog](https://ai-engineering.com), 2026.

[6] Anthropic, "Building Effective AI Agents," anthropic.com, Dec. 2024.

[7] Anthropic, "Claude Memory Architecture," [Claude Docs](https://claude.ai/docs), 2026.

[8] Zep, "Long-Term Memory for AI Agents," getzep.com, 2026.

[9] M. Hashimoto, "My AI Adoption Journey," mitchellh.com, Feb. 2026.

[10] A. Ng, "Agentic Design Patterns," [DeepLearning.AI](https://deeplearning.ai), 2024-2026.

Source Method. *This document is an independent synthesis of public engineering materials, academic research, and production case studies. It is not affiliated with or endorsed by Anthropic, Mem0, Snowflake, or any other organization. All code examples are adapted for pedagogical clarity. Diagrams are original.*